
```
>>> a = ('cat', 'dog', 'moose')
>>> b = ('cat', 'dog', 'moose')
>>> id(a), id(b)
(37111992, 37112136)
❶ >>> id(a) == id(b)
False
>>> hash(a), hash(b)
(-3478972040190420094, -3478972040190420094)
❷ >>> hash(a) == hash(b)
True
```

The tuples referred to by `a` and `b` have different identities ❶, but their identical values mean they'll have identical hashes ❷. Note that a tuple is hashable if it contains only hashable items. Because you can use only hashable items as keys in a dictionary, you can't use a tuple that contains an unhashable list as a key. Enter the following into the interactive shell:

```
>>> tuple1 = ('cat', 'dog')
>>> tuple2 = ('cat', ['apple', 'orange'])
>>> spam = {}
❶ >>> spam[tuple1] = 'a value'
❷ >>> spam[tuple2] = 'another value'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unhashable type: 'list'
```

Notice that `tuple1` is hashable ❶, but `tuple2` contains an unhashable list ❷ and so is also unhashable.

Containers, Sequences, Mapping, and Set Types

The words *container*, *sequence*, and *mapping* have meanings in Python that don't necessarily apply to other programming languages. In Python, a *container* is an object of any data type that can contain multiple other objects. Lists and dictionaries are common container types used in Python.

A *sequence* is an object of any container data type with ordered values accessible through integer indexes. Strings, tuples, lists, and bytes objects are sequence data types. Objects of these types can access values using integer indexes in the index operator (the `[` and `]` brackets) and can also be